The Gomez Character–Line Minimization Theorem: Existence, Bounds, and Uncomputability of Minimal Program Representations

Kye Gomez
Swarms
kye@swarms.world

Abstract

How short can a program be? For a programming language L over an alphabet Σ and a target function or class f, we define the character minimum $C_{\min}(f,L)$, the line minimum $L_{\min}(f,L)$, and the *Gomez Minimum Program Representation*: the Pareto set of the problem of minimizing both characters and lines subject to semantic equivalence. The *Gomez Character–Line Minimization Theorem* states that (i) both minima exist and the Pareto front is non-empty, finite, and has at most $\ell_1 - L_{\min} + 1$ points; (ii) $C_{\min}$ lies between $\lceil (K_U(f) - c_L)/\log_2 |\Sigma| \rceil$ and $\lceil (K_U(f)+1)/\log_2 B_L \rceil + d_L$, where K_U is Kolmogorov complexity and B_L is the radix of L's literals; (iii) $C_{\min}$ is uncomputable and the set of character-minimal programs is immune; and (iv) in every line-collapsible language $L_{\min} = 1$, and forcing a program onto one line costs at most $\alpha e(p^*) + \beta$ extra characters. In Python that cost is exactly 8 characters plus one per newline or escaped character. We give three algorithms: exact front enumeration for decidable fragments, a semantic dynamic program, and a test-guided reducer. Experiments compute exact minima for 1.94 million polynomial functions and measure 12 Python functions and classes. The best-known programs are 74.8% shorter than canonical code, and the one-line construction meets the theorem's bound with equality on all 48 multi-line programs tested.

1 Introduction

Lines of code are among the oldest software metrics [6], and character counts are what code golf, JavaScript minifiers, and token budgets for code-generating language models all try to drive down. Behind these practices sits a basic question: *for a given function or class, is there a theoretical minimum number of characters and lines needed to implement it, and can we know it?*

For strings, Kolmogorov complexity answers this: the length of the shortest description on a universal machine [7, 15, 4, 9]. Programming languages add three complications. First, programs are strings over a finite alphabet of characters, not bits. Second, programs must be equivalent *as behaviors*, not identical as outputs, and for classes the behavior is an interface observed over time. Third, the number of lines is a second objective that can conflict with the first.

Contributions. (1) A formalization of character and line minimality for functions and classes, with equivalence relative to an interface and a domain, and the *Gomez Minimum Program Representation* (GMPR) as a Pareto set (§2). (2) The *Gomez Character–Line Minimization Theorem* (§3): existence and finiteness, a two-sided Kolmogorov bound, uncomputability and immunity, and a line-collapse law with an exact cost for CPython. (3) Algorithms (§4): exact front enumeration, a semantic dynamic program, and GOMEZREDUCE, a reducer whose upper bounds are certified by testing. (4) Experiments (§6): exact minima for every polynomial expressible in at most 14 characters of a small language, and a 12-task Python benchmark.

2 Formal Setting

Definition 1 (Language)**.** A *programming language* is a triple $L = (\Sigma, \mathrm{Valid}_L, \mathscr{E}_L)$: a finite alphabet Σ with $|\Sigma| \geq 2$ and $\backslash\mathtt{n} \in \Sigma$; a decidable set $\mathrm{Valid}_L \subseteq \Sigma^*$ of syntactically valid programs; and a partial evaluation map $\mathscr{E}_L$ that assigns each valid program its semantic object (a partial function, or a class).

Definition 2 (Semantic equivalence)**.** Fix an *interface I* (a binding name and call shape for functions, or a constructor, method names, and observable attributes for classes) and a domain D of arguments. For a function, $\mathscr{E}_L(p) \cong_D f$ iff for every $x \in D$ the two calls both diverge, or both return values of equal type and value, or both raise exceptions of equal type and arguments. For a class, $\mathscr{E}_L(p) \cong_D f$ iff every finite trace of interface operations with arguments in D produces the same sequence of observations (returns, exceptions, attribute reads). We write $\cong$ when D is the full domain. Reflection (source text, file names, line numbers) is not observable.

Let $\mathrm{Impl}_L(f) = \{p \in \mathrm{Valid}_L : \mathscr{E}_L(p) \cong f\}$; f is *L-implementable* if $\mathrm{Impl}_L(f) \neq \emptyset$. Let $|p|$ be the character length of p and $\lambda(p) = 1 + \mathrm{count}(p, \backslash\mathtt{n})$ its line count (written $L(p)$ in the original statement of the problem).

Definition 3 (Character and line minima)**.**

$$C_{\min}(f,L) = \min_{p \in \Sigma^*} \{\, |p| \;:\; \mathscr{E}_L(p) \cong f \,\}, \qquad L_{\min}(f,L) = 1 + \min_{p \in \Sigma^*} \{\, \mathrm{count}(p, \backslash\mathtt{n}) \;:\; \mathscr{E}_L(p) \cong f \,\}.$$

For $\ell \geq 1$, the *line-constrained minimum* is $C^{[\ell]}(f,L) = \min\{|p| : p \in \mathrm{Impl}_L(f),\ \lambda(p) \leq \ell\}$.

Definition 4 (Gomez Minimum Program Representation)**.** The GMPR of f in L is the solution set of the bi-criterion problem

$$\begin{aligned} &\text{Minimize } |p| && \text{(character count)} \\ &\text{Minimize } \lambda(p) = 1 + \mathrm{count}(p, \backslash\mathtt{n}) && \text{(line count)} \\ &\text{subject to } p \in \Sigma^*, \quad \mathscr{E}_L(p) \cong f, \end{aligned} \tag{1}$$

that is, the set $\mathrm{GMPR}(f,L)$ of Pareto-optimal programs: no $p' \in \mathrm{Impl}_L(f)$ has $|p'| \leq |p|$ and $\lambda(p') \leq \lambda(p)$ with one inequality strict. Its image $F(f,L) = \{(|p|, \lambda(p)) : p \in \mathrm{GMPR}(f,L)\} \subset \mathbb{N}^2$ is the *Gomez front*.

Definition 5 (Description complexity of a behavior)**.** Fix a universal machine U that takes a description $b \in \{0,1\}^*$ and an input. Then $K_U(f) = \min\{|b| : U(b,\cdot) \cong f\}$. By the invariance theorem [9], changing U changes K_U by at most an additive constant.

Definition 6 (Literal radix; line-collapsibility)**.** L has *literal radix* B_L if there is a sub-alphabet $\Lambda \subseteq \Sigma \setminus \{\backslash\mathtt{n}\}$ with $|\Lambda| = B_L \geq 2$ and a fixed program context into which any $w \in \Lambda^*$ can be inserted and read as data. For example, decimal integer literals give $B_L = 10$, and Python string literals over printable ASCII without quote and backslash give $B_L = 93$. L is *line-collapsible with cost* (α, β) if there is a computable map τ on Valid_L such that $\tau(p)$ contains no $\backslash\mathtt{n}$, $\mathscr{E}_L(\tau(p)) \cong \mathscr{E}_L(p)$, and $|\tau(p)| \leq |p| + \alpha\, e(p) + \beta$, where $e(p)$ counts the occurrences in p of a fixed finite set of escape-relevant characters that includes $\backslash\mathtt{n}$.

3 The Gomez Character–Line Minimization Theorem

Theorem 1 (Gomez Character–Line Minimization Theorem)**.** *Let L be a Turing-complete language over a finite alphabet Σ with $|\Sigma| \geq 2$ and $\backslash\mathtt{n} \in \Sigma$, and let f be L-implementable. Then:*

(i) ***Existence and finiteness.*** *$C_{\min}(f,L)$ and $L_{\min}(f,L)$ exist and are attained. The Gomez front is a non-empty finite staircase $F(f,L) = \{(c_1,\ell_1), \ldots, (c_k,\ell_k)\}$ with $C_{\min}(f,L) = c_1 < \cdots < c_k$ and $\ell_1 > \cdots > \ell_k = L_{\min}(f,L)$, where $k \leq \ell_1 - L_{\min}(f,L) + 1 \leq C_{\min}(f,L) - L_{\min}(f,L) + 2$. The set $\mathrm{GMPR}(f,L)$ is finite.*

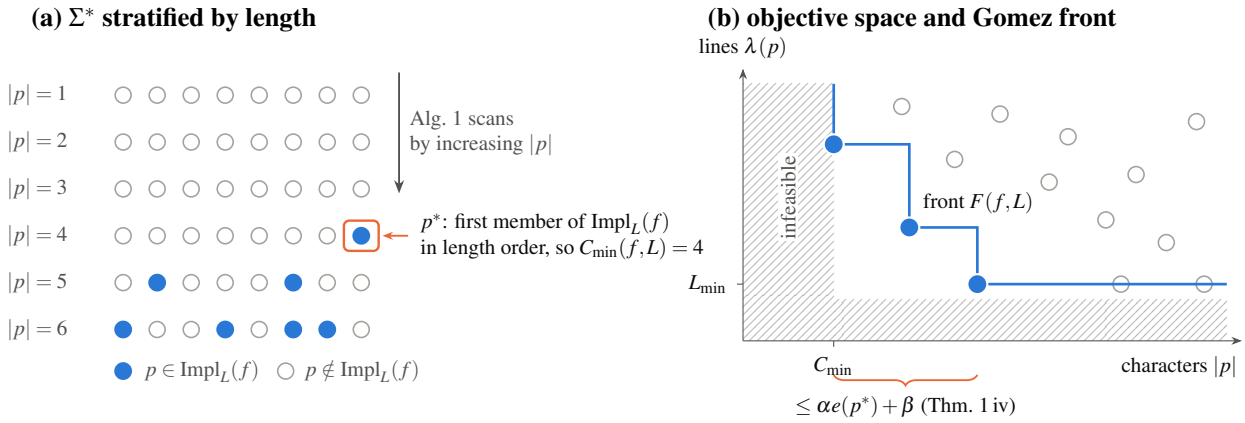


Figure 1: The minimization problem. (a) Programs ordered by length; the first program equivalent to f fixes $C_{\min}$. (b) Each feasible program is a point; the Gomez front is the staircase of non-dominated points. It begins at $C_{\min}$ and ends at $L_{\min}$, and Theorem 1(iv) bounds its width when $L_{\min} = 1$.

(ii) ***Kolmogorov sandwich.*** *For every universal U there are constants c_L, d_L, depending on L and U but not on f, such that*

$$\left\lceil \frac{K_U(f) - c_L}{\log_2 |\Sigma|} \right\rceil \le C_{\min}(f,L) \le \left\lceil \frac{K_U(f)+1}{\log_2 B_L} \right\rceil + d_L. \tag{2}$$

(iii) ***Uncomputability and immunity.*** *No algorithm computes $p \mapsto C_{\min}(\mathscr{E}_L(p), L)$ on Valid_L. The set $M_L = \{p \in \mathrm{Valid}_L : |p| = C_{\min}(\mathscr{E}_L(p), L)\}$ of character-minimal programs is* immune*: it is infinite but has no infinite computably enumerable subset.*

(iv) ***Line collapse.*** *If L is line-collapsible with cost (α, β), then $L_{\min}(f,L) = 1$, and for every character-minimal p^**

$$C_{\min}(f,L) \le C^{[1]}(f,L) \le C_{\min}(f,L) + \alpha\, e(p^*) + \beta \le (1+\alpha) C_{\min}(f,L) + \beta. \tag{3}$$

If $\alpha = \beta = 0$, the front is the single point $(C_{\min}(f,L), 1)$: one program minimizes both objectives.

Proof. *(i)* $\mathrm{Impl}_L(f)$ is non-empty, so $\{|p| : p \in \mathrm{Impl}_L(f)\}$ and $\{\lambda(p) : p \in \mathrm{Impl}_L(f)\}$ are non-empty subsets of $\mathbb{N}$ and have least elements by well-ordering. Among the character-minimal programs, let p_1 have the fewest lines, ℓ_1. Nothing dominates p_1, so the front is non-empty. Any point (c, ℓ) with $\ell \ge \ell_1$ other than (c_1, ℓ_1) is dominated by p_1. Hence every front point has $\ell \in [L_{\min}, \ell_1]$. Two front points cannot share a line count, so $k \le \ell_1 - L_{\min} + 1$. Since every newline is a character, $\ell_1 \le c_1 + 1$. Each front point is realized only by strings of one length c, and there are finitely many of those, so $\mathrm{GMPR}(f,L)$ is finite. Ordering the points by increasing c forces strictly decreasing ℓ. The point with $\ell = L_{\min}$ is the shortest program among those with the fewest lines, and it is non-dominated.

(ii) Lower bound. Encode $p \in \Sigma^n$ by reading it as an n-digit base-$|\Sigma|$ numeral and writing that number in exactly $m = \lceil n \log_2 |\Sigma| \rceil$ bits. Since $\log_2 |\Sigma| \ge 1$, the map $n \mapsto m$ is strictly increasing, so m determines n and the code is decodable. Let ι_L be a self-delimiting index of a machine that decodes such a code and runs an L-interpreter. For a character-minimal p^*, $U(\iota_L \mathrm{code}(p^*), \cdot) \cong \mathscr{E}_L(p^*) \cong f$, so $K_U(f) \le |\iota_L| + C_{\min} \log_2 |\Sigma| + 1$. Rearranging and taking $c_L = |\iota_L| + 1$ gives the left side of (2); the ceiling is valid because $C_{\min}$ is an integer. *Upper bound.* Let b^* be a shortest description and N the integer with binary representation $1b^*$, so $N < 2^{K_U(f)+1}$. Written in base B_L, N takes at most $\lceil (K_U(f)+1)/\log_2 B_L \rceil$ literal characters. Let d_L be the length of a fixed L-context that decodes such a literal, strips the leading 1, and simulates U on the result. Since L is Turing-complete, such a context exists. This gives the right side.

(iii) Suppose $W \subseteq M_L$ is infinite and computably enumerable. For $n \in \mathbb{N}$, let P_n be the L-program that enumerates W until it finds some q with $|q| > n$ (one exists because W is infinite and each length has finitely

many strings) and then behaves as $\mathscr{E}_L(q)$ via a self-interpreter. Only the decimal numeral for n varies, so $|P_n| \le e_L + \lceil \log_{10}(n+1) \rceil$ for a constant e_L. Choose n with $e_L + \lceil \log_{10}(n+1) \rceil < n$. Then $P_n \in \mathrm{Impl}_L(\mathscr{E}_L(q))$ gives $C_{\min}(\mathscr{E}_L(q)) \le |P_n| < n < |q| = C_{\min}(\mathscr{E}_L(q))$, a contradiction. Thus M_L is immune; it is infinite because L implements infinitely many distinct functions and each has a minimal program. If $C_{\min}$ were computable, $M_L = \{p : |p| = C_{\min}(\mathscr{E}_L(p))\}$ would be decidable, hence an infinite c.e. subset of itself. This is a Berry-paradox argument, consistent with Blum's theorem on minimal programs [3, 13].

(iv) Let p^* be character-minimal. Then $\tau(p^*)$ has no newline, so $\lambda(\tau(p^*)) = 1$ and $L_{\min} = 1$. Since $\tau(p^*) \in \mathrm{Impl}_L(f)$, $C^{[1]} \le |\tau(p^*)| \le C_{\min} + \alpha e(p^*) + \beta$, and $e(p^*) \le |p^*|$. If $\alpha = \beta = 0$, then $\tau(p^*)$ attains both individual minima and so dominates every other point. □

Lemma 1 (CPython is line-collapsible with $(\alpha, \beta) = (1, 8)$)**.** *For a module-level CPython program p, choose the quote $q \in \{\texttt{"}, \texttt{'}\}$ that occurs less often in p. Let $\tau(p) =$* `exec(`$q\,\mathrm{esc}_q(p)\,q$`)`*, where* esc_q *rewrites* `\` *as* `\\`*, q as* `\`q*, and newline as the two characters* `\n`*. Then $\tau(p)$ is newline-free, defines the same module bindings with the same observable behavior, and*

$$|\tau(p)| = |p| + 8 + \mathrm{count}(p, \texttt{\textbackslash n}) + \mathrm{count}(p, \texttt{\textbackslash}) + \mathrm{count}(p, q).$$

Proof. The literal decodes to p exactly. At module scope, `exec` with default arguments executes in the module's own namespace, so every binding p creates is created identically. The constant 8 is `|exec("")|`, and each escaped character adds one backslash. □

The original statement of the problem wrote the Kolmogorov bound as an equality, $C_{\min} = \lceil K_L(f) / \log_2 |\Sigma| \rceil$. That equality holds exactly whenever $|\Sigma|$ is a power of two, for the canonical bit-level machine U_L that reads b as a sequence of fixed-width character codes of $\lceil \log_2 |\Sigma| \rceil$ bits and runs the decoded string as an L-program. For 7-bit ASCII, for instance, $K_{U_L}(f) = 7\,C_{\min}(f, L)$. For any other universal reference machine, invariance turns the equality into the two-sided bound (2). With $|\Sigma| = 128$ and $B_L = 93$, the two slopes differ by a factor of only $7/\log_2 93 \approx 1.07$.

Corollary 1 (Counting)**.** *For every $m \in \mathbb{N}$, fewer than $|\Sigma|^m$ distinct behaviors have $C_{\min}(f, L) < m$. For example, more than 99% of the 2^{2^k} Boolean functions of k inputs need at least $\lfloor 2^k/7 \rfloor - 1$ characters in any 128-character language.*

Corollary 2 (Minimal programs are nearly incompressible)**.** *For every character-minimal p^*, $K_U(p^*) \ge (|p^*| - d_L - 1) \log_2 B_L - c'_L$. Asymptotically, a minimal Python program carries at least $\log_2 93 \approx 6.54$ bits of incompressible information per character.*

Proof. $K_U(f) \le K_U(p^*) + O(1)$, because f can be computed from a description of p^*. Combine this with the right side of (2). □

Remark 1 (When $L_{\min} > 1$). Line collapse is a property of the language. In Whitespace the line feed is an instruction symbol: every I/O instruction begins `[Tab][LF]`, so every program with output has $\lambda \ge 2$; C preprocessor directives also need their own lines. Brainfuck treats newline as a comment, so its minimal programs never contain one: $(\alpha, \beta) = (0, 0)$.

4 Algorithms

Theorem 1(iii) rules out a general procedure for computing $C_{\min}$. It leaves two practical routes. One is exact computation where equivalence is decidable (Algorithms 1 and 2). The other is upper bounds certified by testing (Algorithm 3).

Because candidates are enumerated in increasing length, the first hit has length $C_{\min}$. Every later hit with strictly fewer lines than all earlier hits is non-dominated, so the algorithm outputs exactly $F(f, L)$ in $O(|\Sigma|^N)$

Algorithm 1 EXACTFRONT: the Gomez front when $\cong$ is decidable

Require: reference program q; decision procedure for $\cong$; collapse cost (α, β)

$F \leftarrow \emptyset$; $\ell_{\text{best}} \leftarrow \infty$; $N \leftarrow |q|$ ▷ q itself witnesses $C_{\min} \le |q|$
for $n = 1, 2, \ldots$ **while** $n \le N$ **do**
  for $p \in \Sigma^n$ in lexicographic order **do**
    if $p \in \text{Valid}_L$ **and** $p \cong q$ **and** $\lambda(p) < \ell_{\text{best}}$ **then**
      if $F = \emptyset$ **then** $N \leftarrow n + \alpha\, e(p) + \beta$ ▷ Thm. 1(iv): a 1-line program exists by length N
      $F \leftarrow F \cup \{(n, \lambda(p))\}$; $\ell_{\text{best}} \leftarrow \lambda(p)$
      if $\ell_{\text{best}} = 1$ **then return** F
return F

equivalence tests. When the semantics is compositional, enumeration can work on *meanings* instead of strings, as in observational-equivalence pruning from enumerative synthesis [17, 1]. Algorithm 2 does this; it is the procedure behind the exact results in §6.

Algorithm 2 SEMANTICDP: exact $C_{\min}$ for every behavior up to length $n_{\max}$

Require: grammar G with nonterminals X; compositional semantics $[\![\cdot]\!]$ with canonical forms

for $n = 1$ **to** $n_{\max}$ **do**
  for each nonterminal X (in dependency order) **do**
    $M[X][n] \leftarrow \{[\![t]\!] : t \text{ a terminal string of length } n \text{ derivable from } X\}$
    for each rule $X \to Y\, op\, Z$ and each split $i + |op| + j = n$ **do**
      $M[X][n] \leftarrow M[X][n] \cup \{op(a,b) : a \in M[Y][i],\ b \in M[Z][j]\}$ ▷ keep one witness per meaning
  for each $m \in M[S][n]$ not seen before **do** $C_{\min}(m) \leftarrow n$

Outside decidable fragments, Algorithm 3 computes upper bounds. It first applies transformations that provably preserve semantics, then deletes characters under a differential-testing oracle.

Algorithm 3 GOMEZREDUCE: certified-by-testing upper bound on $C_{\min}$

Require: program p; differential-testing oracle $\mathscr{O}(c) = [c \text{ compiles warning-free and } c \cong_D p \text{ on tests}]$

$p \leftarrow$ ALPHARENAME(STRIPDOCSTRINGS(p)) ▷ Phase 1: provably safe
$p \leftarrow$ re-indent to one space per level; delete every space and redundant `( )` pair whose removal leaves the AST unchanged
$p \leftarrow$ join lines (`\n` $\mapsto$ `;` or suite-onto-header) whenever the AST is unchanged
$p \leftarrow$ DDMIN($p, \mathscr{O}$) ▷ Phase 2: delta debugging [18]
repeat for $k = 8, \ldots, 1$: delete any length-k substring s of p if $\mathscr{O}(p \setminus s)$
until no deletion succeeds
re-run the AST-invariant joins; **return** p ▷ $C_{\min} \le |p|$ if $\mathscr{O}$ is exhaustive on D

Phase 1 changes only whitespace, parentheses, and names, and each step is checked against the abstract syntax tree, so it preserves semantics by construction. Phase 2 deletes arbitrary characters and is sound only to the extent that the test oracle is: §6 shows a case where it overfits. Line collapse (Lemma 1) takes six lines:

```
def tau(p):
    """Newline-free equivalent of a module-level CPython program (Lemma 1)."""
    if "\n" not in p:
        return p
    q = '"' if p.count('"') <= p.count("'") else "'"
    body = p.replace("\\", "\\\\").replace(q, "\\" + q).replace("\n", "\\n")
    return "exec(" + q + body + q + ")"
```

5 Code Examples

A function. Euclid's algorithm, written canonically, reduced automatically, and in its best-known form. Note what the reference semantics excludes: `from math import gcd as f` is shorter, but it is *not* equivalent, because the reference returns $f(4,-6) = -2$ while `math.gcd` returns 2.

```
def f(a, b):                        # canonical: 124 chars, 5 lines
    """Greatest common divisor by Euclid's algorithm."""
    while b != 0:
        a, b = b, a % b
    return a

def f(a,b):                         # GomezReduce output: 40 chars, 3 lines
 while b:a,b=b,a%b
 return a

f=lambda a,b:b and f(b,a%b)or a      # best known: 31 chars, 1 line
```

A class. A stack with interface $\{\texttt{push}, \texttt{pop}, \texttt{peek}, \texttt{size}, \texttt{empty}\}$. The canonical version takes 347 characters and 20 lines. Inheriting from `list` reuses `pop` and `__len__`, so even the exception messages on an empty stack are unchanged, and the class is observationally equivalent under Definition 2:

```
class S(list):push=list.append;peek=lambda s:s[-1];size=list.__len__;empty=lambda s:s==[]
```

A non-trivial front. Python has no expression-level exception handling, so a function that catches an exception has a two-point front. The 3-line program is shorter; the 1-line program is τ of it, and its cost of $45+8+2 = 55$ characters matches Lemma 1 exactly:

```
def f(s):
 try:return int(s)
 except:return 0
exec("def f(s):\n try:return int(s)\n except:return 0")
```

6 Experiments

All code ran on CPython 3.13. A program counts as valid only if it compiles with every warning promoted to an error. Characters are bytes of ASCII source; lines are $1 + \text{count}(p, \texttt{\textbackslash n})$.

6.1 Exact minima in a decidable fragment

Language P has alphabet $\{\texttt{x}, \texttt{1}, \texttt{2}, \texttt{+}, \texttt{-}, \texttt{*}, \texttt{(}, \texttt{)}\}$ ($|\Sigma| = 8$) and the usual precedence grammar. Each program denotes an integer polynomial, so $\cong$ is decidable by normal form. We ran Algorithm 2 to $n_{\max} = 14$ in 18 s. Every one of the 1,942,453 polynomials expressible in at most 14 characters received its exact $C_{\min}$ (Figure 2a). At length 14 alone there are 198,527,152 valid programs but only 1,598,072 distinct meanings, about 124 programs per behavior. Valid programs grow about 4.2× per character and distinct behaviors only about 2.7×, so minimal programs become an ever-smaller fraction of all programs. Exact minima can be surprising: $C_{\min}(x^2+2x+1) = 9$ via `1+x*(x+2)`, two characters shorter than the factored `(x+1)*(x+1)`, while $C_{\min}(x^2+4x+4) = 11$ via `(x+2)*(x+2)`. Figure 2b gives the exact character complexity of the integers 0 to 400. It is erratic and non-monotone: $C_{\min}(10) = 4$ (`11-1`) but $C_{\min}(11) = 2$. The maximum is 13, reached at $359 =$ `2+21*(21-2*2)` and $369 =$ `(1+2)*(1+122)`. Allowing `x` never shortened a constant (cross-checked up to length 11).

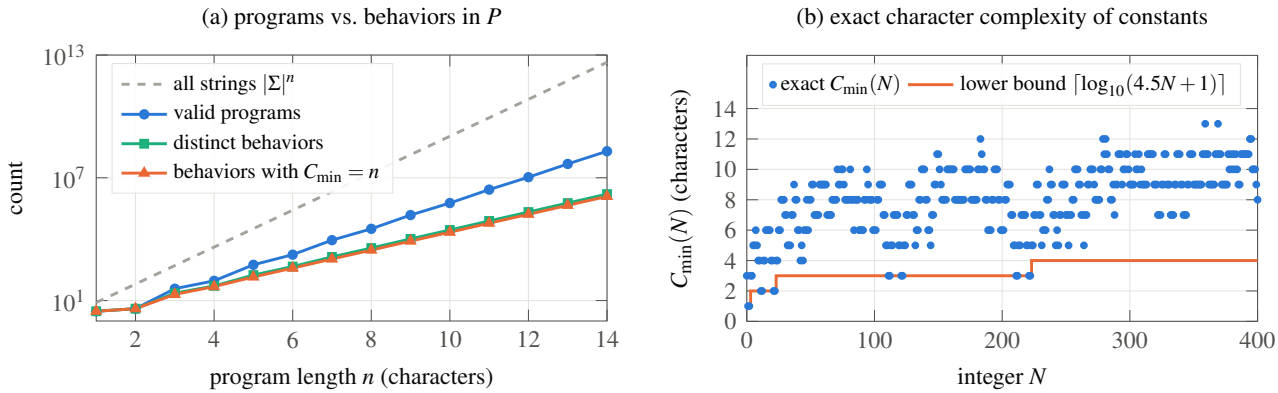


Figure 2: Exact results in the decidable fragment P (Algorithm 2, $n_{\max} = 14$). (a) Valid programs outgrow distinct behaviors, so the number of programs per behavior rises from 1 to 124. (b) $C_{\min}(N)$ for $0 \le N \le 400$, with the bound implied by the fact that the largest value writable in n characters is $22\cdots2$.

Table 1: Python benchmark, as characters / lines. "Best" columns are the shortest verified programs we know with at least two lines and with exactly one line; the last column is the best-known Gomez front. These are upper bounds: true minima are uncomputable (Theorem 1 iii). †Equivalent only on the test domain (see Finding 5).

Task	Kind	Canonical	GomezReduce	Best ≥2-line	Best 1-line	Best-known front F
`factorial`	function	150 / 6	53 / 4	50 / 4	28 / 1	(28,1)
`fibonacci`	function	148 / 6	57 / 4	57 / 4	41 / 1	(41,1)
`gcd`	function	124 / 5	40 / 3	40 / 3	31 / 1	(31,1)
`is_prime`	function	201 / 10	95 / 7	72 / 6	47 / 1	(47,1)
`palindrome`	function	211 / 9	91 / 6	28 / 2	21 / 1	(21,1)
`fizzbuzz`	function	353 / 13	169 / 8	74 / 2	68 / 1	(68,1)
`flatten`	function	214 / 9	96 / 6	62 / 4	57 / 1	(57,1)
`collatz`	function	228 / 10	77 / 7†	61 / 4	45 / 1	(45,1)
`transpose`	function	207 / 6	102 / 3	37 / 2	31 / 1	(31,1)
`safe_int`	function	155 / 6	56 / 3	45 / 3	55 / 1	(45,3), (55,1)
`stack`	class	347 / 20	204 / 7	204 / 7	89 / 1	(89,1)
`account`	class	406 / 15	197 / 6	188 / 6	194 / 1	(188,6), (194,1)
Total characters		2,744	1,237			691 (best overall)

6.2 Python functions and classes

We specified 12 tasks: ten functions and two classes, `stack` and `account`. Each task has an interface, a domain generator, and a canonical PEP-8 reference implementation. The oracle compares types, values, and exceptions on 35–500 inputs per function, or 60 random traces of up to 25 operations per class. Every candidate below passed, and each GomezReduce output also passed a second test suite generated from a fresh seed. Table 1 and Figure 3 summarize the results.

Findings. (1) Phase 1 of GomezReduce, which is provably safe, removes 54.8% of canonical characters. Phase 2 adds only 0.1 points over 12,097 oracle calls, but on `gcd` it found `while b:` and matched the hand-written minimum. (2) Best-known programs are 74.8% shorter than canonical ones and reach this through semantic rewrites: recursion, comprehensions, inheritance, and library reuse. Character deletion cannot find these, which is the practical face of Theorem 1(iii). (3) In 10 of the 12 tasks, the shortest known program already has one line, as Theorem 1(iv) suggests. The two exceptions need `try/except` or `raise`, and their fronts have two points (Figure 4). (4) On all 48 multi-line candidates (36 distinct programs), $|\tau(p)|$ equaled the Lemma 1

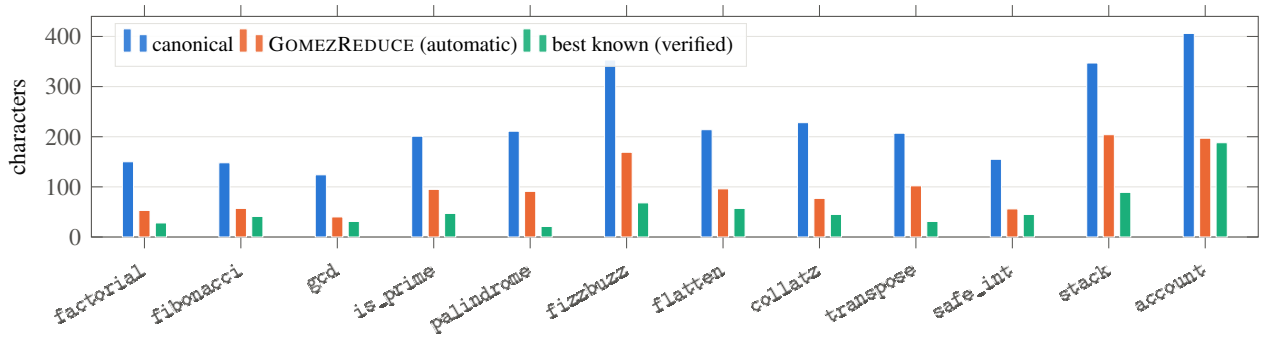


Figure 3: Character counts per task. The automatic reducer removes 54.9% of canonical characters in total. Best-known programs remove 74.8%, mostly through semantic rewrites that character deletion cannot discover.

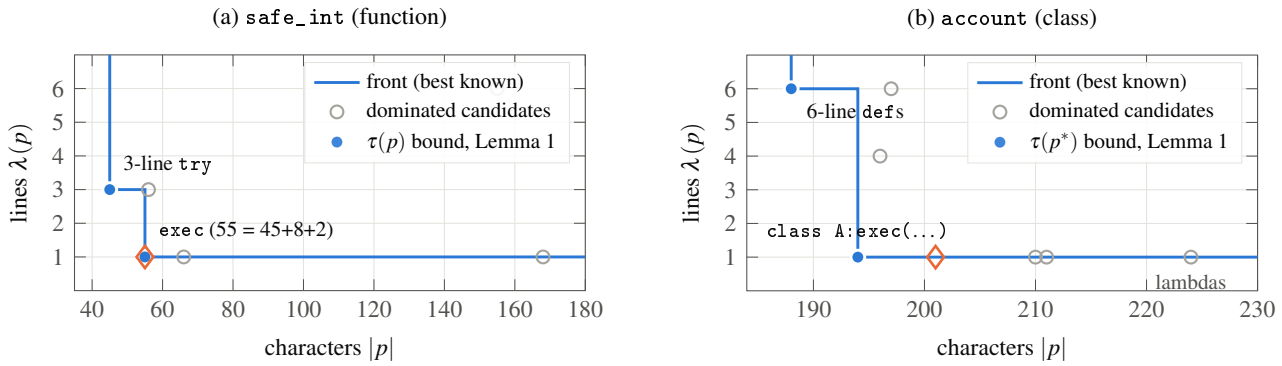


Figure 4: Best-known Gomez fronts for the two tasks whose fronts have more than one point. Python has no expression-level `try` or `raise`, so the shortest known program has several lines, and moving to one line costs 10 and 6 characters, both within the bound of Theorem 1(iv). The `account` canonical program (406, 15) and its collapse (428, 1) lie off-scale; the point at (196, 4) applies `exec` to one method only.

formula exactly. For `account`, a hand-written collapse that applies `exec` to the class body (194 characters) beats the generic τ bound of 201, so the bound is an upper bound, not the one-line minimum. (5) Testing can overfit. On `collatz`, Phase 2 replaced `n//2` with `n/2`. The result agrees with the reference on the whole test domain ($n \leq 5000$, and on fresh seeds), but floats lose exactness past 2^{53}: at $n = 2^{53} - 4$ the reference takes 596 steps and the reduced program 547. Every GomezReduce bound is therefore a bound for $\cong_D$, not for $\cong$.

Compressibility. Corollary 2 predicts that minimization removes redundancy. Concatenating the 12 programs at each stage, order-0 entropy rose from 4.28 bits per character (canonical) to 4.90 (reduced) to 5.06 (best known), and `zlib` from 2.80 to 3.38 to 4.65 (`lzma`: 2.87, 3.54, 5.03). Compressor overhead partly confounds the `zlib` trend on small corpora. All values stay below the asymptotic 6.54-bit floor, as expected when d_L dominates.

7 Related Work

Description complexity for strings is due to Kolmogorov, Solomonoff, and Chaitin [7, 15, 4]; Li and Vitányi [9] treat invariance, counting, and incompressibility. Blum proved that minimal programs in acceptable programming systems cannot be effectively listed [3], and Schaefer surveys the minimal-index sets [13]. Undecidability of semantic equivalence follows from Rice's theorem [12] and Turing [16], and Levin's universal search [8] motivates length-ordered enumeration. Superoptimization searches for the shortest instruction sequence equivalent to a given one [10, 14]. Enumerative and syntax-guided synthesis prune candidates by observational equiva-

lence [2, 17, 1, 5]. Delta debugging and C-Reduce minimize test inputs under an oracle [18, 11]; GOMEZREDUCE adapts them to program minimization. To our knowledge, earlier work has not treated line count as a second objective or characterized when it is free, cheap, or essential.

8 Discussion and Conclusion

The theorem separates what is knowable about minimal programs from what is not. Minima exist, the Gomez front is a finite staircase, and $C_{\min}$ is pinned to $\Theta(K_U(f))$. Lines are almost free: in any line-collapsible language one line suffices, at an additive cost that is exactly computable for CPython. The character minimum itself cannot be computed, and minimal programs cannot be recognized in bulk. Three limitations apply. The constants c_L, d_L for production languages are large, so (2) is informative only asymptotically. Equivalence in practice is relative to a domain, and the `collatz` case shows that this matters. Finally, $C_{\min}$ ignores running time: `f=lambda n:n if n<2 else f(n-1)+f(n-2)` (38 characters) is equivalent to our 41-character best-known Fibonacci but exponential-time, so our oracle's 2-second budget rejects it. Our bounds are thus implicitly time-bounded; a Levin-style Kt variant of $C_{\min}$ is a natural next step. For code-generating agents, the results suggest applying provably safe transformations first (Phase 1 captures most of the automatic gain), then semantic rewriting under explicit domain contracts, and treating claims of minimality as unprovable.

References

[1] A. Albarghouthi, S. Gulwani, Z. Kincaid. Recursive program synthesis. In *Proc. CAV*, LNCS 8044, pp. 934–950, 2013.

[2] R. Alur, R. Bodik, G. Juniwal, et al. Syntax-guided synthesis. In *Proc. FMCAD*, pp. 1–8, 2013.

[3] M. Blum. On the size of machines. *Information and Control*, 11(3):257–265, 1967.

[4] G. J. Chaitin. On the length of programs for computing finite binary sequences. *Journal of the ACM*, 13(4):547–569, 1966.

[5] S. Gulwani, O. Polozov, R. Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 4(1–2):1–119, 2017.

[6] M. H. Halstead. *Elements of Software Science*. Elsevier North-Holland, 1977.

[7] A. N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7, 1965.

[8] L. A. Levin. Universal sequential search problems. *Problems of Information Transmission*, 9(3):265–266, 1973.

[9] M. Li, P. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*, 4th ed. Springer, 2019.

[10] H. Massalin. Superoptimizer: A look at the smallest program. In *Proc. ASPLOS II*, pp. 122–126, 1987.

[11] J. Regehr, Y. Chen, P. Cuoq, E. Eide, C. Ellison, X. Yang. Test-case reduction for C compiler bugs. In *Proc. PLDI*, pp. 335–346, 2012.

[12] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the AMS*, 74(2):358–366, 1953.

[13] M. Schaefer. A guided tour of minimal indices and shortest descriptions. *Archive for Mathematical Logic*, 37(8):521–548, 1998.

[14] E. Schkufza, R. Sharma, A. Aiken. Stochastic superoptimization. In *Proc. ASPLOS*, pp. 305–316, 2013.

[15] R. J. Solomonoff. A formal theory of inductive inference, Part I. *Information and Control*, 7(1):1–22, 1964.

[16] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proc. London Mathematical Society*, s2-42(1):230–265, 1936.

[17] A. Udupa, A. Raghavan, J. V. Deshmukh, S. Mador-Haim, M. M. K. Martin, R. Alur. TRANSIT: Specifying protocols with concolic snippets. In *Proc. PLDI*, pp. 287–296, 2013.

[18] A. Zeller, R. Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002.